

RoK4 Reward Structure

작성일: 2026-07-15 최종 업데이트: 2026-08-03

이 문서는 RoK4-Isaac-Velocity-Flat-v0 task의 현재 reward 구조와 reward function 설정을 정리한다. 현재 reward는 Isaac Lab G1 velocity task 구조를 출발점으로 RoK4 ADAPT actuator 좌표, direct velocity command, standing transition에 맞게 조정한 flat walking baseline이다. 현재 reference는

2026-07-24_19-34-26_symmetry_aug_nojumps2_swing_roll100_fresh/model_9999.pt 이며 experimental Yunho Symmetry ADAPT v1 으로 기록한다. 이 checkpoint는 domain randomization과 policy observation noise를 조정하기 전의 비교 기준이며, 이후 실험에서도 보존한다.

아래 경로에서 RoK4: 는 `${ROK4_ROOT}` 를, Isaac Lab: 은 `${ISAACLAB_ROOT}` 를 기준으로 한 상대경로를 뜻한다. 두 변수는 각 사용자가 clone한 저장소 root를 가리킨다.

```
export ROK4_ROOT="${HOME}/rok4_lab"
export ISAACLAB_ROOT="${HOME}/IsaacLab"
```

Observation noise, reset state randomization, physics DR의 상세 범위와 적용 주기는 docs/rok4_randomization_and_noise_ko.rst 를 기준 문서로 사용한다.

관련 파일

역할	파일
RoK4 flat task 설정	RoK4: <code>.../config/rok4/flat_env_cfg.py</code>
RoK4 ADAPT transmission/actuator	RoK4: <code>.../assets/robots/rok4_adapt.py</code>
RoK4 actuator action/observation	RoK4: <code>.../velocity/mdp/actions.py, observations.py</code>
RoK4 actuator reward 함수	RoK4: <code>.../velocity/mdp/rewards.py</code>
RoK4 domain randomization 설정	RoK4: <code>.../config/rok4/domain_randomization_cfg.py</code>
공통 locomotion reward term 기본값	Isaac Lab: <code>.../velocity/velocity_env_cfg.py</code>
locomotion 전용 reward 함수	Isaac Lab: <code>.../velocity/mdp/rewards.py</code>
Isaac Lab 공통 reward 함수	Isaac Lab: <code>isaac_lab/envs/mdp/rewards.py</code>
mdp namespace 연결	Isaac Lab: <code>.../velocity/mdp/_init_.py</code>

Actuator interface와 reward 연결

현재 reward는 단순히 joint 값을 actuator라고 이름만 바꾼 것이 아니다. action target, 실제 상태, PD torque가 각각 ADAPT 행렬을 통과하며, reward는 그 runtime actuator 값 또는 mapped joint target을 명시적으로 사용한다.

```

rok4.py
└─ 링크 길이, actuator gain/action scale/limit 설정
  └─ rok4_adapt.py
      └─ RoK4AdaptTransmission:  $J/J^{-1}/J^T/J^{-T}$ 
          └─ RoK4AdaptActuator: actuator PD와  $\tau_{\psi}/\tau_q$  계산
              │
              └─ +-> mdp/actions.py
                  │   raw action ->  $\psi_{\text{target}}$  ->  $q_{\text{target}}$ 
                  │
                  └─ +-> mdp/observations.py
                      │    $q/\dot{q}$  ->  $\psi/\dot{\psi}$  observation
                      │
                      └─ +-> mdp/rewards.py
                          └─  $\tau_{\psi}$ ,  $\dot{\psi}$ ,  $\ddot{\psi}$  penalty
                              └─ actuator torque/velocity limit 초과 penalty
                                  └─ clipped raw actuator action-rate penalty
                                      └─  $q_{\text{target}}$  joint-position-limit penalty
                                          └─ flat_env_cfg.py의 RoK4RewardsCfg
                                              └─ func + weight + params를 RewardTerm으로 구성

```

rok4_adapt.py, actions.py, observations.py, rewards.py 는 모두 rok4_lab 안의 로컬 파일이며, Isaac Lab 원본 source 를 수정하지 않는다. Isaac Lab의 부모 class와 mdp 함수는 import/상속/re-export해서 사용한다.

구조 요약

RoK4FlatEnvCfg 는 환경 설정이고, RoK4RewardsCfg 는 reward term 묶음이다. Reward 구조는 두 가지 관계로 나누어 보면 될 것 같다.

첫 번째는 config class의 상속/포함 관계이다.



```
└─ action_rate_l2
└─ second_action_rate_l2
```

두 번째는 각 reward term이 실제 계산 함수를 참조하는 관계이다.

```
RoK4FlatEnvCfg
└─ rewards = RoK4RewardsCfg()
    └─ RoK4RewardsCfg extends RewardsCfg
        └─ 각 reward term은 RewardTermCfg/RewTerm
            └─ func=mdp.xxx
                └─ RoK4 로컬 mdp 함수
                    source/rok4_tasks/.../velocity/mdp/rewards.py
                └─ Isaac Lab 공통 mdp 함수
                    IsaacLab/source/isaacsim/isaacsim/envs/mdp/rewards.py
                └─ locomotion velocity 전용 mdp 함수
                    IsaacLab/source/isaacsim_tasks/.../velocity/mdp/rewards.py
```

즉 RoK4FlatEnvCfg -> RoK4RewardsCfg 는 상속이 아니라 포함/사용 관계이고, RoK4RewardsCfg -> RewardsCfg 는 상속 관계이다. mdp/rewards.py 는 부모 클래스가 아니라 실제 reward 계산 함수가 들어 있는 함수 모음이다. RoK4는 로컬 rok4_tasks...velocity.mdp 를 import하며, 이 로컬 mdp는 Isaac Lab의 기존 locomotion mdp를 다시 export하고 RoK4 전용 reward 함수만 추가한다.

실제로는 다음처럼 이해하면 된다.

```
RoK4RewardsCfg는 RewardsCfg를 상속받아서 기본 reward term들을 물려받는다.
RoK4에 필요한 reward는 mdp.xxx 함수를 직접 지정해 새로 정의하거나 override한다.
RewardTermCfg/RewTerm은 mdp.xxx 함수, weight, params를 하나의 reward term으로 묶는다.
```

파일 배치 기준은 다음과 같다.

```
flat_env_cfg.py
└─ 어떤 reward term을 쓸지, weight를 얼마로 둘지, 어떤 body/joint에 적용할지 결정

velocity/mdp/rewards.py
└─ reward raw value를 어떻게 계산할지 정의
```

따라서 joint_action_target_pos_limits, actuator_torques_l2, actuator_vel_l2, actuator_acc_l2, actuator_velocity_limits, actuator_torque_limits, action_rate_l2, second_action_rate_l2 처럼 RoK4 전용

weighting을 쓰는 계산식은 velocity/mdp/rewards.py 에 두고, weight를 얼마로 사용할지는 RoK4RewardsCfg 에 둔다. 이 term은 이전 action 두 개가 필요하므로 reset 직후 첫 두 policy step에서는 penalty를 0으로 둔다.

Action-rate reward의 action 기준

현재 RoK4Lab의 action_rate_l2 와 second_action_rate_l2 는 env.action_manager.action 을 기준으로 현재 action 과 history의 차분을 만든다. 이 action buffer는 scale 적용 전 clipped raw policy action이며, reward도 이 raw 차분의 weighted 제곱합을 계산한다. Action scale은 적용하지 않지만 Hip Pitch/Knee에 해당하는 index 2,3,8,9 의 제곱 오차에는 RoK4 전용 완화 weight 0.5 를 적용한다.

다만 RoK4FlatPP0RunnerCfg 에서 clip_actions = 1.0 을 명시하므로 Isaac Lab train.py 와 play.py 를 통해 실행할 때는 RslRlVecEnvWrapper 가 action을 먼저 [-1, 1] 로 clamp한다. 따라서 reward가 보는 action은 wrapper에서 [-1, 1] 로 잘린 raw actuator action이다.

```
actor output
-> clip_actions = 1.0
-> clipped_raw_actuator_action
├ last_action observation = clipped_raw_actuator_action
├ action_rate_l2 / second_action_rate_l2
└ scaled_actuator_offset = clipped_raw_actuator_action * ROK4_ACTUATOR_ACTION_SCALE
    └ psi_target = psi_default + scaled_actuator_offset
        └ q_target = J * psi_target
            └ joint_action_target_pos_limits compares q_target with soft_joint_pos_limits
```

여기서 q_target 은 PhysX position drive에 직접 입력되는 값이 아니라 explicit actuator model의 입력이다.

RoK4AdaptActuator.compute() 가 q_target 과 현재 joint state를 actuator 좌표로 변환해 tau_psi 를 계산·제한하고, tau_q = J^{-T} tau_psi 로 바꾼 joint effort만 PhysX에 전달한다. Actuator torque는 최대값의 90%에서 제어 및 reward 기준으로 제한하고, PhysX effort_limit_sim 은 factor를 적용하지 않은 joint mechanical maximum을 최종 solver 안전 상한으로 사용한다.

기존 IsaacGym RoK4 코드는 actuator_actions_raw * action_scale 으로 만든 scaled actuator_actions 를 두 action-rate penalty에 넘겼다. 현재 RoK4Lab은 이 부분만 G1 방식으로 바꾸어 clipped raw action 차분을 사용한다.

ROK4_ACTUATOR_ACTION_SCALE 값은 Gym의 좌우 다리 [0.4, 0.5, 1.25, 1.5, 0.75, 0.75] 와 torso 0.4 를 그대로 유지하지만 actuator target 생성에만 사용한다. Observation의 last_action 과 두 action-rate reward가 같은 clipped raw action 좌표를 사용한다.

이 변경은 tensor 크기만 보면 이전 joint-space policy와 같지만 action, joint/actuator position, velocity의 의미가 다르다. 기존 joint-space checkpoint를 resume하면 잘못된 좌표 의미로 학습되므로 actuator-interface checkpoint는 반드시 새 학습에서

생성한다.

Action target의 default_joint_pos 도 Gym의 활성 gait-ready pose와 동일하다. Hip pitch는 -0.0924 rad, knee pitch는 0.345 rad, ankle pitch는 -0.253 rad 이며 나머지 관절은 0.0 rad 이다. 이 joint pose를 psi_default = $J^{-1} q_{\text{default}}$ 로 변환한 값이 actuator action 중심이며, joint_deviation_* reward는 계속 원래 joint default pose를 기준으로 한다.

Reward와 Domain Randomization의 구분

RoK4RewardsCfg 와 부모 RewardsCfg 는 reward term을 정의한다. 반면 foot friction, restitution, base/upper/lower body mass, base/upper/lower COM, reset pose 같은 domain randomization 값은 domain_randomization_cfg.py 에서 관리한다.

두 설정은 학습 결과에 모두 영향을 주지만 역할은 다르다.

구분	파일	역할
Reward	flat_env_cfg.py 의 RoK4RewardsCfg 및 self.rewards.xxx	policy가 어떤 행동을 더 좋게 볼지 점수 함수를 정의
DR	domain_randomization_cfg.py	물성, 질량, COM, 초기상태 perturbation을 바꿔 다양한 상황에서 버티게 함

예를 들어 발 미끄러짐은 feet_slide reward가 penalty로 줄이고, foot-ground 마찰 계수는 domain_randomization_cfg.py 의 ROK4_STATIC_FRICTION_RANGE 와 ROK4_DYNAMIC_FRICTION_RATIO 가 정한다. 따라서 미끄러짐 문제를 볼 때는 reward 와 DR을 함께 확인해야 하지만, 코드상 관리 위치는 분리되어 있다.

현재 material event는 모든 robot collision shape를 G1/Digit식 nominal $\mu_{\text{static}}=0.8, \mu_{\text{dynamic}}=0.6$ 으로 먼저 고정한다. 그 뒤 Foot material DR이 환경마다 $\mu_{\text{static}} \sim \text{Uniform}(0.5, 0.9)$ 를 표본화하고 $\mu_{\text{dynamic}} = 0.75 * \mu_{\text{static}}$ 으로 계산한다. 같은 환경의 좌우 Foot은 동일한 material bucket을 사용하므로 마찰 DR 자체가 좌우 gait 비대칭을 만들지 않으며, dynamic friction은 항상 static friction 이하로 유지된다. Play와 Teleop에서는 이 범위를 사용하지 않고 nominal 0.8/0.6 으로 고정해 checkpoint를 재현성 있게 비교한다.

관절 물성은 학습 scene 생성 시 각 environment/joint에 대해 ROK4_STATIC_FRICTION, ROK4_VISCOUS_FRICTION, ROK4_ARMATURE 의 nominal 값을 각각 독립적인 $\text{Uniform}(0.8, 1.2)$ 배율로 scale한다. 이는 PhysX joint property DR이며 actuator-space PD의 ROK4_ACTUATOR_KP/KD gain DR이 아니다. Play와 Teleop에서는 이 joint-physics DR event를 제거해 nominal 관절 물성을 사용한다.

Reward와 PPO entropy/KL의 구분

Reward term은 환경의 RewardManager 가 행동의 점수를 계산하는 항목이다. 반면 PPO의 entropy와 KL은 rsl_rl optimizer가 policy distribution을 업데이트할 때 사용하는 학습 지표다. 따라서 entropy_coef 나 desired_kl 은 RoK4RewardsCfg 의 reward weight가 아니다.

Reward와 PPO 지표 비교

항목	현재 값/위치	역할
reward weight	flat_env_cfg.py 의 RewTerm(weight=...)	환경 행동의 positive reward 또는 penalty 크기를 결정

항목	현재 값/위치	역할
entropy_coef	rs_l_rl_ppo_cfg.py 의 0.002	Gaussian policy의 exploration entropy를 유지하려는 optimizer 압력을 결정
desired_kl	rs_l_rl_ppo_cfg.py 의 0.01	old/new policy 차이를 기준으로 adaptive learning rate를 조절

RoK4 로컬 scripts/rs_l_rl/rok4_ppo.py 는 adaptive schedule에서 이미 계산되는 mini-batch KL을 모아 TensorBoard의 Loss/kl``(iteration 평균)과 ``Loss/kl_max``(iteration 최대)로 기록한다. 현재 5 learning epochs와 4 mini-batches를 사용하므로 iteration당 20개 KL 값을 집계한다. 이 로깅을 위해 Isaac Lab 또는 설치된 ``rs_l_rl 파일을 수정하지 않는다.

Reward 계산 방식

Isaac Lab의 RewardManager 는 각 reward term의 raw value를 계산한 뒤 weight 를 곱하고, 모든 term을 합산한다.

```
total_reward =
    sum(weight_i * reward_function_i(env, params_i))
```

따라서 positive weight는 행동을 유도하고, negative weight는 penalty로 작동한다.

RoK4 전용 Reward Terms

아래 term들은 RoK4RewardsCfg 에서 직접 정의하거나 부모 RewardsCfg 의 term을 RoK4용으로 override한 항목이다.

term	function	weight	params	의미
termination_penalty	mdp.is_terminated	-200.0	없음	episode가 termination되면 큰 penalty를 준다.
track_lin_vel_xy_exp	mdp.track_lin_vel_xy_yaw_frame_exp	1.0	command_name="base_velocity", std=0.5	yaw-aligned robot frame에서 x/y 선속도 command를 추종하게 한다.
track_ang_vel_z_exp	mdp.track_ang_vel_z_world_exp	1.0	command_name="base_velocity", std=0.5	world frame yaw angular velocity command를 추종하게 한다.
feet_air_time	mdp.feet_air_time_touch_down_biped	0.5	feet: L_Foot_Link, R_Foot_Link; threshold=0.65	정확히 한 발이 first contact가 된 step에 완료된 air-time을 최대 0.65 까지 한 번 지급한다. 계속 한 발을 들거나 지지하는 동안에는 반복 지급하지 않는다.
no_jumps	mdp.desired_contacts	-2.0	feet: L_Foot_Link, R_Foot_Link; force threshold=1.0 N	최근 contact-force history에서 양발 모두 threshold를 넘는 접촉이 없을 때만 raw penalty 1 을 반환한다. 정상 single stance와 toe-off는 허용하지만 양발 동시 flight phase를 억제한다.
feet_slide	mdp.feet_slide	-0.2	feet: L_Foot_Link, R_Foot_Link	지면 접촉 중인 발이 미끄러지는 것을 줄인다.
feet_flat_orientation_l2	mdp.feet_flat_orientation_l2	None (현재 비활성)	좌우 Foot body quaternion과 contact sensor	합수는 접촉 중인 발의 local +Z 축을 world frame으로 회전한 뒤 X/Y 성분 제곱합을 계산한다. 현재는 world-up 수평 제약이 toe-off와 향후 경사면 적응을 방해하지 않도록 Reward Manager에 등록하지 않는다.
feet_swing_roll_l2	mdp.feet_swing_roll_l2	-1.0	좌우 Foot body quaternion과 contact sensor	공중에 있는 발의 local +Z 법선을 각 Foot 자체의 yaw frame으로 변환하고 lateral 성분의 제곱만 계산한다. 따라서 swing-foot의 안쪽/바깥쪽 roll은 억제하지만 pitch, yaw, toe-off와 지지 발 자세는 제한하지 않는다.
feet_stance_width_l2	mdp.feet_stance_width_l2	None (현재 비활성)	목표 0.21 m, hard minimum 0.18 m; 직진 이동 command에서만 활성화	합수는 좌우 Foot 위치 차이를 base yaw frame으로 회전해 lateral width를 계산한다. 0.21 m 초과 폭은 coefficient 0.5 로 억제하고, 0.18 m 미만 또는 발 교차는 coefficient 5.0 으로 강하게 억제한다. 현재 RoK4RewardsCfg 에서는 term을 None 으로 두어 Reward Manager에 등록하지 않는다.
feet_lateral_separation_l2	mdp.feet_lateral_separation_l2	-2.0	minimum width 0.16 m; feet 순서는 Left, Right	좌우 Foot 위치 차이를 base yaw frame으로 회전한 뒤 signed lateral width y_left - y_right 를 유지한다. relu(0.16 - signed_width)^2 만 반환하므로 정상 0.21 m 폭과 넓은 폭은 제한하지 않고, 좁아질수록 penalty가 증가하며 좌우 Foot이 교차해 부호

term	function	weight	params	의미
				가 바뀌면 더 크게 작동한다. Command mask 없이 standing과 moving 모두에 적용된다.
stand_still_joint_deviation_l1	mdp.stand_still_joint_deviation_l1	-0.2	ROK4_JOINT_ORDER 전체 13관절, base_velocity.is_standing_env mask	RoK4 command generator가 standing으로 지정된 환경에서만 실제 joint position과 default joint position 차이의 절댓값 합을 penalty로 반환한다. 기존 Gym의 defaultPosStanding 에 대응한다.
dof_pos_limits	mdp.joint_pos_limits	-1.0	ROK4_JOINT_ORDER 에 포함된 전체 13관절, preserve_order=True	각 관절이 USD hard limit의 95%로 계산된 soft joint limit을 넘는 양을 합산해 억제한다.
joint_action_target_pos_limits	mdp.joint_action_target_pos_limits	-0.001	actuator_pos action의 mapped joint target과 ROK4_JOINT_ORDER 의 전체 13관절	Gym의 penalty_action_limits 에 대응한다. ADAPT mapping 뒤 목표 관절 위치가 95% soft limit을 넘는 양을 합산한다.
joint_deviation_hip	mdp.joint_deviation_l1	-0.05	.*_Hip_Yaw_Joint, .*_Hip_Roll_Joint	RoK4의 회전된 hip joint frame에서는 lateral foot placement가 이름상 yaw/roll 두 축의 조합으로 생성된다. -0.1 에서 절반으로 완화하여 flat_orientation_l2=-5.0 으로 상체 기울임은 억제하면서 다리가 옆으로 내딛을 자유를 준다. 완전히 끄지는 않아 과도한 hip 편차와 넓은 stance를 계속 억제한다.
joint_deviation_hip_pitch	mdp.joint_deviation_l1	-0.01	.*_Hip_Pitch_Joint	전후진 보폭을 만드는 핵심 관절을 yaw/roll과 같은 강도로 묶지 않으면서, default pose에서 과도하게 벗어나 다리 전체를 크게 휘두르는 전략을 약하게 억제한다. Swing phase나 무릎 굽힘을 직접 판정하는 reward는 아니다.
joint_deviation_torso	mdp.joint_deviation_l1	-0.1	Torso_Yaw_Joint	torso yaw가 default pose에서 과도하게 벗어나지 않게 한다.
actuator_acc_l2	mdp.actuator_acc_l2	-1.0e-8	전체 RoK4 actuator; index 2,3,8,9 weight 0.5	Isaac Lab의 joint acceleration을 J^{-1} 로 actuator acceleration으로 변환한다. 실제 rad/s^2 값이며 Gym의 undivided velocity difference와는 다르다.
actuator_torques_l2	mdp.actuator_torques_l2	-2.0e-6	전체 RoK4 actuator; index 2,3,8,9 weight 0.5	ADAPT explicit actuator가 torque-limit clip 뒤 보관한 τ_{psi} 의 weighted squared sum이다.
actuator_vel_l2	mdp.actuator_vel_l2	-1.0e-4	전체 RoK4 actuator; index 2,3,8,9 weight 0.5	$\dot{\psi} = J^{-1} \dot{q}$ 의 weighted squared sum으로 Gym penalty_joint_vel 에 대응한다.
actuator_velocity_limits	mdp.actuator_velocity_limits	-0.001	velocity_limit_factor=0.9를 적용한 actuator velocity limits	$\text{abs}(\dot{\psi})$ 가 설정된 velocity limit을 넘는 양을 합산한다.
actuator_torque_limits	mdp.actuator_torque_limits	-1.0e-5	torque_limit_factor=0.9를 적용한 actuator torque limits	clip 전 요청 τ_{psi} 가 설정된 torque limit을 넘는 양을 합산해 saturation 요구를 표시한다.
action_rate_l2	mdp.action_rate_l2	-0.005	clipped raw action 전체 13차원; index 2,3,8,9 weight 0.5	$a_t - a_{t-1}$ 의 weighted 제곱합을 줄인다. Action scale은 적용하지 않는다.
second_action_rate_l2	mdp.second_action_rate_l2	-0.0005	clipped raw action 전체 13차원; index 2,3,8,9 weight 0.5	raw action의 2차 차분, 즉 $a_t - 2a_{t-1} + a_{t-2}$ 의 weighted 제곱합으로 action jerk를 약하게 완화한다. reset 직후 첫 두 policy step은 history가 부족하므로 0으로 처리한다.

Gym과 Lab의 action-limit 신호 차이

Gym과 현재 Lab 구현 모두 actuator action을 ADAPT 변환한 mapped joint target을 검사한다. 현재 Lab의 RoK4ActuatorPositionAction.processed_actions 가 $q_{\text{target}} = J \psi_{\text{target}}$ 을 보관하므로 joint_action_target_pos_limits 는 실제 PD command에 대응하는 joint target을 검사한다.

Gym과 Lab의 torque/velocity 신호 차이

기존 Gym 코드와 현재 Lab 코드는 모두 ADAPT transmission 변환 전 actuator torque/velocity를 penalty에 사용한다. 현재 Lab custom actuator는 applied_actuator_effort 를 직접 노출하고 joint velocity를 J^{-1} 로 변환한다. 따라서 torque/velocity reward 좌표계와 0.5 weighting은 Gym과 동일하다. Acceleration만 현재 Lab에서 $J^{-1} \ddot{q}$ 의 실제 acceleration을 사용하므로 Gym의 $\dot{\psi}_t - \dot{\psi}_{t-1}$ 과 정의가 다르다.

부모 RewardsCfg에서 상속받고 RoK4에서 수정한 Terms

아래 term들은 RewardsCfg 에서 기본으로 제공되며, RoK4FlatEnvCfg.__post_init__() 에서 weight나 body/joint 대상이 RoK4에 맞게 수정된다.

term	function	current weight	RoK4 설정	의미
lin_vel_z_l2	mdp.lin_vel_z_l2	-0.2	기본 robot root	base가 z 방향으로 튀는 움직임을 줄인다.
ang_vel_xy_l2	mdp.ang_vel_xy_l2	-0.05	기본 robot root	roll/pitch angular velocity를 줄인다.
flat_orientation_l2	mdp.flat_orientation_l2	-5.0	기본 robot root	projected gravity의 X/Y 성분 제곱합으로 몸이 기울어지는 것을 줄인다. -1.0 과 -2.0 실험 뒤 -5.0 으로 크게 강화하여 lateral hopping이 과도한 body roll 전략인지 확인하고, 자연스러운 체중 이동과 속도 추종을 과도하게 제한하는지도 함께 관찰한다.
undesired_contacts	mdp.undesired_contacts	-1.0	Base_Link, Upper_Body_Link; threshold=1.0	발이 아닌 base/upper body 접촉을 penalty로 처리한다.

Reward Function 요약

function	위치	계산 의미
is_terminated	Isaac Lab 공통	termination manager의 terminated flag를 reward term으로 반환한다.
track_lin_vel_xy_yaw_frame_exp	locomotion mdp	yaw frame x/y 선속도 command error에 $\exp(-\text{error} / \text{std}^2)$ 를 적용한다.
track_ang_vel_z_world_exp	locomotion mdp	world z축 angular velocity command error에 $\exp(-\text{error} / \text{std}^2)$ 를 적용한다.
feet_air_time_touchdown_biped	RoK4 local mdp	정확히 한 발이 first contact가 된 순간 $\min(\text{last_air_time}, \text{threshold})$ 를 한 번 반환하며, 작은 command와 양발 동시 touchdown 은 0이 된다.
desired_contacts	Isaac Lab 공통	지정 body 중 하나라도 최근 force history에서 threshold를 넘으면 0, 모두 접촉하지 않으면 1을 반환한다. Negative weight와 함께 양발 flight penalty로 사용한다.
feet_slide	locomotion mdp	접촉 중인 foot의 horizontal linear velocity norm을 penalty로 계산한다.
lin_vel_z_l2	Isaac Lab 공통	base body-frame z velocity squared.
ang_vel_xy_l2	Isaac Lab 공통	base body-frame roll/pitch angular velocity squared sum.
flat_orientation_l2	Isaac Lab 공통	projected gravity의 x/y component squared sum으로 몸 기울기를 penalty화한다.
actuator_torques_l2	RoK4 로컬 mdp	actuator-space applied torque weighted squared sum. Index 2,3,8,9 는 weight 0.5.
actuator_vel_l2	RoK4 로컬 mdp	$J^{-1} \dot{q}$ actuator velocity의 weighted squared sum. Index 2,3,8,9 는 weight 0.5.
actuator_acc_l2	RoK4 로컬 mdp	$J^{-1} \ddot{q}$ actuator acceleration의 weighted squared sum. Index 2,3,8,9 는 weight 0.5.
actuator_velocity_limits	RoK4 로컬 mdp	velocity_limit_factor 를 적용한 actuator velocity limit 초과량.
actuator_torque_limits	RoK4 로컬 mdp	clip 전 actuator torque command가 torque_limit_factor 적용 limit를 넘은 양.
action_rate_l2	RoK4 로컬 mdp	현재 clipped raw action과 이전 action 차이의 weighted squared sum. Index 2,3,8,9 는 weight 0.5.
second_action_rate_l2	RoK4 로컬 mdp	clipped raw action의 2차 차분 weighted squared sum. Index 2,3,8,9 는 weight 0.5.
joint_deviation_l1	Isaac Lab 공통	현재 joint position과 default joint position 차이의 absolute sum.
stand_still_joint_deviation_l1	RoK4 로컬 mdp	base_velocity.is_standing_env 가 true일 때 전체 13관절의 default-pose absolute error sum.
joint_pos_limits	Isaac Lab 공통	soft joint position limit을 넘은 정도를 합산한다.
joint_action_target_pos_limits	RoK4 로컬 mdp	actuator_pos action이 ADAPT mapping한 joint target의 soft joint position limit 초과량.
undesired_contacts	Isaac Lab 공통	지정 body의 contact force가 threshold를 넘은 contact 개수를 penalty로 반환한다.

Command와 Reward의 연결

현재 RoK4 flat command range는 다음과 같다.

```
lin_vel_x = (-0.3, 0.85)
lin_vel_y = (-0.3, 0.3)
ang_vel_z = (-0.6, 0.6)
```

RoK4PeriodicFreezeVelocityCommand 는 episode reset마다 환경 역할을 다음 확률로 표본화한다.

1. 45% mixed: vx, vy, wz 를 모두 표본화한다.

2. 5% standing: episode 동안 exact-zero command를 유지한다.
3. 5% walking: planar command norm 0.15 m/s 이상만 사용하고 freeze하지 않는다.
4. 10% x: $v_x = \pm \text{Uniform}(0.15, 0.30)$ m/s 저속 대칭 전후진만 표본화한다.
5. 5% fast_forward: $v_x = \text{Uniform}(0.30, 0.85)$ m/s 고속 전진만 표본화한다.
6. 10% y: lateral 좌/우 command만 표본화한다.
7. 10% yaw: 시계/반시계 yaw-rate command만 표본화한다.
8. 10% x_yaw: $v_y = 0$ 인 저속 대칭 전후진+회전 command를 표본화한다.

코드에서는 command 축과 직접 대응하는 x/y/yaw/x_yaw 이름을 사용한다. 각 단일 축 역할은 양/음 부호를 50:50 확률로 표본화하며 x_yaw 의 두 부호는 독립적으로 표본화한다. x 와 x_yaw 의 v_x 는 절댓값 0.15~0.30 m/s 로 대칭이고, fast_forward 가 0.30~0.85 m/s 를 별도로 담당한다. v_y 의 최소 절댓값은 0.15 m/s, w_z 는 0.15 rad/s 이다. 4096개 환경에서 각 10% 역할은 평균 약 410개이고, 각 부호는 평균 약 205개이므로 이전 Gym의 방향별 200개 population과 비슷한 표본 규모다.

mixed/x/fast_forward/y/yaw/x_yaw 의 비율 합 90%는 10.0 s cycle과 환경별 $\text{Uniform}(1.5, 3.0)$ s freeze duration을 사용한다. Random initial phase와 독립 duration 때문에 같은 PPO batch에서 moving, standing, transition을 동시에 관측한다. Episode 중 command를 다시 표본화할 때는 역할을 유지하며, reset 때 역할 자체를 다시 무작위로 뽑는다. 부모의 별도 standing 확률은 $\text{rel_standing_envs}=0.0$ 으로 비활성화한다. 작은 non-zero mixed command는 standing으로 바꾸지 않고 연속적인 저속 이동 목표로 유지한다.

여기서 random initial phase는 global simulation 시작 시 한 번만 주는 값이 아니다. 각 environment가 reset될 때마다 freeze phase를 $\text{Uniform}(0, 10)$ s 로 다시 표본화하므로 reset 직후 freeze로 시작할 수도 있다. 학습 episode timeout은 환경별 20 s 이며, early termination이 난 환경만 episode counter, episode role, freeze phase를 다시 시작한다. 예를 들어 global time 3 s 에 reset된 환경은 이후 20초를 생존하면 약 23 s 에 timeout된다. 초기에는 episode counter가 같지만 early reset이 누적되면서 환경별 episode phase가 자연스럽게 달라진다.

부모 task에서 상속한 training push_robot 은 freeze와 다른 환경별 interval timer다. Reset마다 다음 push 시간을 $\text{Uniform}(10, 15)$ s 로 표본화하고, world-frame root x/y velocity에 각각 -0.5~0.5 m/s 를 추가한다. 따라서 push는 이동 command 중에도, exact-zero freeze 중에도, 전환 부근에도 발생할 수 있다. Freeze 중 push가 들어와도 command와 is_standing_env 는 zero/true로 유지되므로 velocity-tracking reward와 stand_still_joint_deviation_l1 은 외란에서 정지 자세로 복원하는 행동을 평가한다. Contact reward와 no_jumps 는 같은 구간의 실제 발 접촉 상태를 그대로 평가한다. Play/Teleop에서는 이 자동 interval event를 끄고 수동 Push Test UI를 사용한다.

Periodic freeze와 standing 역할은 command를 $[0, 0, 0]$ 으로 만드는 동시에 is_standing_env=True 를 설정한다. stand_still_joint_deviation_l1 은 command 크기를 다시 판정하지 않고 이 mask를 직접 사용하여 전체 13관절을 default pose 근처로 유지한다. 현재 weight는 -0.2 이다. -1.0 실험은 안정적인 양발 standing을 만들었지만 feet-air-time 감소와 foot-slide 증가가 관측되어 보행 자유도를 회복하도록 완화했다.

이 command는 `track_lin_vel_xy_exp`, `track_ang_vel_z_exp`, `feet_air_time` 에 직접 영향을 준다. 특히 `feet_air_time_touchdown_biped` 는 x/y command norm이 0.1 이하이면 reward를 0으로 만든다. 즉 거의 정지 명령에서는 stepping reward가 강하게 작동하지 않는다.

`UniformVelocityCommand` 가 반환하는 `[lin_vel_x, lin_vel_y, ang_vel_z]` 는 robot base frame 기준 command다. 따라서 양의 `lin_vel_x` 는 로봇이 현재 바라보는 방향의 전진, 양의 `lin_vel_y` 는 로봇 기준 좌측 이동, 양의 `ang_vel_z` 는 z축 주위 반시계 방향 회전을 뜻한다. 선속도 tracking 함수는 실제 world 선속도에서 roll/pitch를 제외한 yaw 회전만 역변환한 gravity-aligned yaw frame 속도와 x/y command를 비교한다. 반면 현재 각속도 tracking 함수는 command의 z 성분과 world-frame `root_ang_vel_w[:, 2]` 를 비교한다. 평평하고 직립한 상태에서는 두 z축이 거의 같지만, 몸체가 크게 기울면 완전히 동일한 표현은 아니다.

RoK4는 부모 G1-style heading mode를 사용하지 않는다. `heading_command=False`, `rel_heading_envs=0.0`, `heading=None` 으로 설정하고 `ang_vel_z=(-0.6, 0.6)` rad/s 에서 목표 wz 를 직접 표본화한다. 따라서 학습, Gym, Teleop, ROS cmd_vel 과 실물 적용에서 policy가 받는 세 번째 command는 모두 같은 의미의 목표 yaw angular velocity다. Navigation에서 목표 world heading이 필요하다면 policy 외부의 상위 controller가 heading error를 wz 로 변환해 이 동일한 velocity interface에 전달한다.

현재 `feet_air_time` 은 느리고 긴 step을 유도하기 위해 `threshold=0.65 s`, `weight=0.5` 를 사용한다. 정확히 한 발이 first contact가 된 policy step에 그 발의 완료된 `last_air_time` 을 읽고 `min(last_air_time, 0.65)` 를 한 번 반환한다. 양발이 같은 step에 first contact가 되거나 planar command norm이 0.1 m/s 이하이면 0이다. 따라서 최대 pre-dt event 크기는 $0.65 * 0.5 = 0.325$ 이지만, 한 발 지지를 계속 유지하거나 air-time이 0.65 s 를 넘는 동안에는 추가 reward를 받지 않는다. 이 sparse event term의 TensorBoard 값은 이전 dense per-step term과 직접 비교할 수 없다.

`no_jumps` 는 `mdp.desired_contacts` 를 `weight=-2.0` 과 `force threshold=1.0 N` 으로 사용한다. 최근 contact-force history 에서 좌우 Foot 모두 threshold를 넘지 못한 경우에만 raw value 1 을 반환하므로, Reward Manager에서는 해당 양발 flight interval에 음의 점수가 적용된다. 한 발이라도 접촉 중이면 0이므로 정상 single stance와 toe-off는 직접 억제하지 않는다. 또한 이 term만으로 좌우 교대나 최대 single-stance 시간을 강제하지는 않는다. 두 velocity-tracking term의 최대 pre-dt 합 +2.0 을 flight 구간에서 상쇄하도록 -2.0 을 첫 실험값으로 사용한다.

`feet_swing_roll_l2` 는 `weight=-1.0` 로 활성화되어 swing 중 발바닥이 안쪽 또는 바깥쪽으로 말리는 현상을 억제한다. 각 Foot의 local +z 법선을 world frame으로 회전한 뒤 그 Foot 자체의 yaw frame으로 옮기고, lateral 성분 $n_y^2 = \sin^2(\text{roll})$ 만 penalty로 반환한다. 따라서 foot yaw나 pitch가 변해도 roll 성분만 분리한다. Contact sensor의 `current_contact_time` 이 0인 발에만 적용하고 swing 발 수로 평균하므로, 지지 발은 0이며 pitch와 yaw도 직접 제한하지 않는다. 따라서 기존의 비활성 `feet_flat_orientation_l2` 와 달리 toe-off나 fore-aft swing 각도를 평평하게 만들지 않는다.

Contact sensor는 `update_period=0.002 s` 와 `history_length=self.decimation=5` 를 사용한다. 따라서 `no_jumps`, `feet_slide`, `undesired_contacts`, `illegal_body_contact` 처럼 `net_forces_w_history` 를 검사하는 항목은 최근 policy

interval의 5개 physics contact sample을 확인한다. 반면 `feet_air_time_touchdown_biped` 는 `compute_first_contact(env.step_dt)` 와 완료된 `last_air_time` 을 사용한다. 이 contact-force history는 policy observation history와 별개의 buffer이다.

Play 환경은 현재 standing 검증을 위해 `lin_vel_x=0.0` m/s 를 사용하고 lateral/yaw command도 0으로 고정한다. 별도의 Teleop 환경은 긴 resampling interval을 사용하고, gamepad 또는 keyboard의 [`lin_vel_x`, `lin_vel_y`, `ang_vel_z`] 를 동일한 base-frame command buffer에 직접 기록한다. 입력은 학습 범위 $(-0.3, 0.85)$ m/s, $(-0.3, 0.3)$ m/s, $(-0.6, 0.6)$ rad/s 안으로 scale된다. 이는 inference command source만 바꾸며 reward 정의나 학습 checkpoint 자체를 변경하지 않는다. Play와 Teleop에서는 학습 전용 periodic freeze를 비활성화한다.

Termination과 Reward의 연결

RoK4 flat task는 Isaac Lab의 `TerminationsCfg` 를 수정하지 않고 로컬 `RoK4TerminationsCfg` 로 상속한다. 부모의 `time_out` 은 유지하고 `base_contact` 는 비활성화한 뒤 `illegal_body_contact` 를 추가한다.

```
class RoK4TerminationsCfg(TerminationsCfg):
    base_contact = None
    illegal_body_contact = DoneTerm(
        func=mdp.illegal_contact,
        params={
            "sensor_cfg": SceneEntityCfg(
                "contact_forces",
                body_names=ROK4_ILLEGAL_CONTACT_BODY_NAMES,
            ),
            "threshold": 1.0,
        },
    )
```

`ROK4_ILLEGAL_CONTACT_BODY_NAMES` 에는 `Base_Link`, `Upper_Body_Link` 와 좌우 `Hip_Yaw`, `Hip_Roll`, `Thigh`, `Calf`, `Ankle_Pitch`, `Ankle_Roll` link가 포함된다. 정상 접촉 body인 `L_Foot_Link` 와 `R_Foot_Link` 만 제외된다. 선택된 body 중 하나라도 contact force가 1.0 N 을 넘으면 episode가 종료된다.

`mdp.is_terminated` 는 non-timeout termination을 감지하므로 `illegal_body_contact` 가 발생하면 `termination_penalty=-200.0` 가 함께 작동한다. 반면 부모에서 상속한 정상 `time_out` 에는 이 penalty가 적용되지 않는다. 여러 body가 동시에 접촉해도 termination flag는 boolean이므로 termination penalty는 한 번만 적용된다. Contact sensor 는 상대 물체 종류를 구분하지 않은 net contact force를 사용하므로 self-collision으로 선택 body에 큰 접촉력이 생겨도 종료 될 수 있다.

현재 설계 의도

현재 reward는 다음 목표를 동시에 가진다.

- 1. command velocity를 따라간다.
- 2. upright 자세를 유지한다.
- 3. 한 발씩 드는 biped stepping pattern을 만든다.
- 4. 양발이 동시에 공중에 뜨는 flight phase를 줄이되 정상 single stance는 허용한다.
- 5. standing과 moving 모두에서 좌우 Foot의 signed lateral separation 하한만 두어 다리 교차를 억제한다.
- 6. foot-flat orientation과 목표 stance-width 함수는 구현되어 있지만 현재 비활성이다.
- 7. 발 미끄러짐을 줄인다.
- 8. torque, joint acceleration, action rate, second action rate를 줄여 움직임을 부드럽게 한다.
- 9. ankle limit, hip/torso deviation을 제한한다.
- 10. zero command에서 전체 13관절을 default standing pose 근처로 유지한다.
- 11. Foot를 제외한 body 접촉을 실패 종료로 처리한다.

튜닝 시 우선 확인할 항목

현상	먼저 볼 항목
너무 자주 넘어진다	termination_penalty, illegal_body_contact, self-collision, flat_orientation_l2, 초기 자세
거의 걷지 않고 버틴다	track_lin_vel_xy_exp, command range, feet_air_time
발이 많이 미끄러진다	feet_slide, foot collision, friction, contact sensor
양발이 동시에 공중에 뜨는 점프가 반복된다	no_jumps, contact-force threshold/history, feet_air_time 의 single-stance 보상 크기
발날 또는 발끝으로 착지한다	ankle actuator gain/target, Foot body frame, contact sensor를 먼저 확인한다. feet_flat_orientation_l2 는 world-up 기준 함수이므로 평지 진단 실험에서만 선택적으로 활성화한다.
다리를 과도하게 벌리거나 교차한다	feet_lateral_separation_l2 의 signed width와 penalty를 먼저 확인한다. 이 term은 교차/최소 폭만 다루며 넓은 stance는 제한하지 않는다. 넓은 stance 문제에는 joint_deviation_hip 와 비활성 feet_stance_width_l2 를 별도로 검토한다.
관절이 떨린다	action_rate_l2, second_action_rate_l2, actuator_acc_l2, actuator PD gain, action scale
토크가 과도하다	actuator_torques_l2, actuator torque limit, action scale
관절이 limit 근처로 간다	dof_pos_limits, joint_action_target_pos_limits, 해당 joint의 default pose, action scale, USD hard limit